

RCMultiBus – Übersicht und Funktionsweise

Version: 1.05

Datum: 11.01.2021

Autor: Wolfgang Haring e.U.

Firmenbuchnummer: FN 312421 v (Landesgericht Wr. Neustadt)

Adresse: Kammanngasse 7-9A/8, A-2700 Wiener Neustadt



image1.png



image2.png

Allgemeines

Der **RCMultiBus** ist ein übergeordnetes Protokoll für alle 3-Draht-Datenbusse, die im RC-Bereich verwendet werden. Ein Slave-Gerät, das RCMultiBus unterstützt, muss in der Lage sein, alle Busarten inklusive Protokoll, Baudrate und Polarität zu interpretieren. Ein Master-Gerät hingegen muss lediglich eine Busart erzeugen können.

Die aktuell unterstützten Busarten sind:

- IBUS
- SBUS
- EX-Bus
- CPPM

Je nach Busart können proportionale und/oder digitale Informationen empfangen werden. Jeder proportionale Kanal wird dabei immer in digitale Schaltfunktionen (permanent und im Speicher) umgewandelt.

Busarten und Kanalwerte

Jeder Kanalwert wird gespeichert und kann individuell auf Servo- oder Motor-Ausgänge gemappt werden. Zusätzlich wird die Bewegung jedes Kanals überwacht, woraus ein Bitfeld generiert wird. Dieses Bitfeld besitzt drei Dimensionen:

- **Richtung:** Top (oben) / Bottom (unten)
- **Bewegungsumfang:** Full (voll) / Half (halb)
- **Speicherstatus:** Memory (im Speicher) / Permanent (dauerhaft)

Channel	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
CH1	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
CH2	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
...							
CHx	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full

Beispiele für Bitfeld-Flags

- **Beispiel 1:** Kanal 4 wird auf +30 % bewegt → Flag *CH4 Permanent/Top/Half* wird gesetzt.
- **Beispiel 2:** Kanal 4 wird innerhalb einer halben Sekunde zurückbewegt → Flag *CH4 Permanent/Top/Half* wird gelöscht, Flag *CH4 Memory/Top/Half* wird gesetzt.

- **Beispiel 3:** Kanal 1 wird auf -100 % bewegt → Flag *CH1 Permanent/Bottom/Full* wird gesetzt.
- **Beispiel 4:** Kanal 1 wird nach einer Sekunde zurückbewegt → Flag *CH1 Permanent/Bottom/Full* wird gelöscht, Flag *CH1 Memory/Bottom/Full* wird gesetzt.

Multiswitch-Systeme (Futaba, Graupner)

Multiswitch-Systeme sind langsamer, da die Signale multiplexartig nacheinander übertragen werden und hauptsächlich Knöpfe oder Schalter (keine Sticks) verwenden. Daher können die Zustände *“short”* und *“half”* hier nicht verwendet werden.

Channel	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
CH1	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
CH2	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
...							
CHx	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full

SUMD und SUMDV3

Die Verwendung von SUMD oder SUMDV3 hängt von der Version des Senders/Empfängers sowie der Firmware des Empfängers ab.

- **SUMD:** Funktioniert analog zu IBUS, SBUS, EX-Bus und CPPM (siehe oben).
- **SUMDV3:** Überträgt ein Feld sogenannter „digitaler Schalter“, die von RCMultiBus-Modulen empfangen und gespeichert werden. Hier können sowohl kanalabhängige Schalter wie bei IBUS, SBUS, EX-Bus, CPPM als auch SUMDV3-eigene digitale Schalter verwendet werden.

PWM Standard

Funktioniert ähnlich wie IBUS, SBUS, EX-Bus und CPPM, jedoch mit nur einem Kanal (CH1).

UART-Einstellungen

- Standard: 38.400 Baud, 8 Bytes, LSB zuerst, keine Parität, 1 Stoppbit, Timeout 100 µs
- Zukünftig: 100.000 Baud (in Planung)

Zentrale zeitgesteuerte Funktionen

Zentrale zeitgesteuerte Funktionen wie Blinker werden durch zwei digitale Funktionen gesteuert:

- Information, ob die Funktion generell ein- oder ausgeschaltet ist
- Impuls, der die Lampe an- und ausschaltet

Um den Blinker zu steuern, genügt es, den Impuls zu setzen. Die generelle Ein-/Aus-Information kann für Mixer oder andere Schalter verwendet werden (z. B. Umschaltung auf eine zweite Konfiguration, wenn Blinker aktiv ist).

Datenstrukturen

Basisfunktionen (Tag 0x01)

Basisfunktionen sind in 5 Bytes codiert:

Offset	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	Wamblinker	Blinker links	Blinker rechts	Fernlichtblitzer	Nebelscheinwerfer	Fernlicht	Abblendlicht	Standlicht
1	Stützbeine hoch	Fünfte Rad	Motor läuft	Starten	Hupen	Hecknebellicht	Bremslicht	Rückfahrlight
2	Blinker rechts Impuls	Rundumleuchte	Tagfahrlight	Abbiegelicht links	Abbiegelicht rechts	Mulde runter	Mulde hoch	Stützbeine runter

Offset	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
3	Intern/RFU	Signalhorn	Gang runter	Gang hoch	Kurvenlicht links	Kurvenlicht rechts	Intern/RFU	Blinker links Impuls
4	Servo 2	Servo 1	Gang 2/2	Gang 1/2	Pad-Level	Rampe runter	Rampe hoch	Intern/RFU

Zusatzfunktionen (Tag 0x05)

Offset	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	CH1 Memory Half Short Rechts	CH1 Memory Half Short Links	CH1 Rechts Voll	CH1 Links Voll	CH1 Memory Voll Lang Rechts	CH1 Memory Voll Lang Links	CH1 Memory Voll Kurz Rechts	CH1 Memory Voll Kurz Links
1	CH2 Memory Voll Lang Rechts	CH2 Memory Voll Lang Links	CH2 Memory Voll Kurz Rechts	CH2 Memory Voll Kurz Links	CH1 Rechts Halb	CH1 Links Halb	CH1 Memory Half Lang Rechts	CH1 Memory Half Lang Links
2	CH2 Rechts Halb	CH2 Links Halb	CH2 Memory Half Lang Rechts	CH2 Memory Half Lang Links	CH2 Memory Half Kurz Rechts	CH2 Memory Half Kurz Links	CH2 Rechts Voll	CH2 Links Voll
...	...							
45	CH31 Memory Half Short Rechts	CH31 Memory Half Short Links	CH31 Rechts Voll	CH31 Links Voll	CH31 Memory Voll Lang Rechts	CH31 Memory Voll Lang Links	CH31 Memory Voll Kurz Rechts	CH31 Memory Voll Kurz Links
46	CH32 Memory Voll Lang Rechts	CH32 Memory Voll Lang Links	CH32 Memory Voll Kurz Rechts	CH32 Memory Voll Kurz Links	CH31 Rechts Halb	CH31 Links Halb	CH31 Memory Half Lang Rechts	CH31 Memory Half Lang Links
47	CH32 Rechts Halb	CH32 Links Halb	CH32 Memory Half Lang Rechts	CH32 Memory Half Lang Links	CH32 Memory Half Kurz Rechts	CH32 Memory Half Kurz Links	CH32 Rechts Voll	CH32 Links Voll

Proportionale Funktionen (Tag 0x41)

Die Werte sind 16-Bit vorzeichenbehaftete Werte im Bereich von -500 bis +500, was einer Anzeige von -100 % bis +100 % entspricht. Die Daten liegen im Little-Endian-Format vor.

CH1 (Gas)	CH2 (Lenkung)	CH3	CH4	...	CH31	CH32
-----------	---------------	-----	-----	-----	------	------

Kommandos

Alle Kommandos beginnen mit 0xA1 oder 0xA2, sind als Tag-Length-Value (TLV) kodiert und enden mit einer XOR-Prüfsumme.

Startbyte	Tag (Kommando)	Länge	Wert (Daten)	Prüfsumme
0xA1 / 0xA2	0x6C	0x03	0x01 0x01 0x01	0xCF

Die Prüfsumme wird als XOR über alle Bytes mit 0xAA als Startwert berechnet.

SET_FCT_DIRECT – Direkte Funktionssteuerung

Mit dieser Funktion können einzelne Funktionen direkt ein- oder ausgeschaltet werden.

Startbyte	Tag (Kommando)	Länge	Wert (Daten)	Prüfsumme
0xA1	0x6C	0x03	0x01 0x03 0x01	0xCD

Datenformat

Byte 1	Byte 2	Byte 3
Datenobjekt (z. B. Basisfunktionen)	Bit-Offset (1–40, z. B. Parklicht, Abblendlicht)	Ein/Aus (1 = Ein, 0 = Aus)

SET_PARAMETER – Parameteränderung

Mit dieser Funktion können Skriptparameter für alle Ausgänge geändert werden, z. B. „Alle Fernlicht-Helligkeitswerte auf 80 % setzen“.

Startbyte	Tag (Kommando)	Länge	Wert (Daten)	Prüfsumme
0xA1	0x76	0x02	0x04 0x50	0x81

SET_DATA – Periodische Datenübertragung

Diese Funktion dient dazu, alle Werte – sowohl proportionale als auch digitale Funktionen – periodisch zu setzen.

Startbyte	Tag der Datenstruktur	Länge	Wert (Daten)	Prüfsumme
0xA2	entsprechender Tag	0x05	0x87 0x80 0x00 0x00 0x00	0xCF